

```
#보유 미보유 리스트
owned = []
not_owned = ['목각 인형', '돌무덤', '물부리', '추억의 펜던트', '어느 날의 기억', '경외심', '상처불이', '작고 나쁜 인형', '억류된 찬송', '밀라르카', '사원증', '순철용 손전등', '휴대용 전지 소켓', '무정: 2017년 1월 10일']
```

결론부터 말씀드리면, 과제에서 요구하는 5가지 필수 조건을 모두 충족하게 잘 작성하셨습니다. 다만, 프로그램을 실행했을 때 치명적인 오류(에러)를 발생시키는 문법적 실수가 몇 가지 있어 이 부분을 수정해야 완벽하게 동작합니다.

번호	항목	충족 여부	설명
1	사용자 메뉴 구성	충족	<pre>while True: 프를 사용했고, 선택 시 break 지나옵니다.</pre>
2	함수 정의 및 분리	충족	<pre>menu_start 총 5개의 함수를 정의했고, 메인 결하게 구성했</pre>

...

- 수정 전: `owned.insert('추억')`
- 수정 후: `owned.append('추억')` (치성, 인슐레이터 부분도 동일하게 수정 필요)

2. menu_end() 함수의 count_item 변수 참조 문제

menu_end() 함수 안에서 count_item 을 계산에 사용하고 있는데, 이 변수는 함수 밖(while 루프 안)에서 선언되었습니다. 파이썬이 전역 변수로 인식해서 당장 에러가 나진 않지만, 매개변수로 전달받아 사용하는 것이 훨씬 안전하고 좋은 코딩 습관입니다.

- 수정 전: `def menu_end():`
- 수정 후: `def menu_end(count_item):` (호출할 때도 `result_light = menu_end(count_item)` 으로 수정)

3. ZeroDivisionError (0으로 나누기 오류) 방지

menu_end() 함수에서 `((value + money) // count_item)` 계산을 수행합니다. 만약 보유한 아이템이 하나도 없어서 count_item 이 0 인 상태로 종료하면 0으로 나누기 에러가 발생하여 프로그램이 비정상 종료됩니다.



수정된 완성 코드

위의 개선 사항들을 반영하여 수정한 코드입니다. 이 코드를 복사해서 테스트해 보시기 바랍니다.

Python



```
#보유 미보유 리스트
owned = []
not_owned = ['목각 인형', '돌무덤', '물부리', '추억의 펜던트', '어느 날의 기억', '경외심', '상처']

#합성 목록
consums = ['추억', '치성', '인슐레이터']
consumed = []

#돈 가치
money = 0
value = 0

# 시작 팩
start_breath = ['목각 인형', '돌무덤', '물부리']
start_bleed = ['경외심', '상처불이', '작고 나뭇 인형']
start_hunger = ['단 이인종', '고수화우', '상피구리', '가슴비어', '가슴비어', '가슴비어']
```

```

# 시작 팩
start_breath = ['목각 인형', '돌무덤', '물부리']
start_bleed = ['경외심', '상처불이', '작고 나뭇 인형']
start_charge = ['사원증', '순찰용 손전등', '휴대용 전지 소켓']

# 반복 방지
first_rep = 0

def menu_start_pack():
    global owned
    global not_owned
    global value
    global first_rep

    if first_rep != 1:
        print(f'시작 아이템 팩을 선택')
        start_pack = input(f"호흡 출혈 충전 중 입력 : ")

        if start_pack == '호흡':
            owned = list(start_breath)
            for item in start_breath:
                not_owned.remove(item)
            value += 150
            first_rep = 1
        elif start_pack == '출혈':
            owned = list(start_bleed)
            for item in start_bleed:
                not_owned.remove(item)
            value += 150
            first_rep = 1
        elif start_pack == '충전':
            owned = list(start_charge)
            for item in start_charge:
                not_owned.remove(item)
            value += 150
            first_rep = 1
        else :
            print(f'잘못된 입력입니다.')
    else:
        print(f'이미 선택하였습니다.')

```

```

def menu_sell_item(sell_item):
    global owned
    global not_owned
    global value
    global money

    if sell_item in owned:
        price = float(input(f'아이템 가격 : '))
        value -= price
        money += price
        owned.remove(sell_item)
        not_owned.append(sell_item)
    else:
        print(f'미보유이거나 존재하지 않는 아이템 입니다.')

def menu_consum_item(consum_item):
    global owned
    global value
    global consums
    global consumed

    if consum_item == '추억':
        if '추억의 펜던트' in owned and '어느 날의 기억' in owned:
            print(f'추억을 합성합니다.')
            owned.append('추억') # insert() 대신 append() 사용
            consums.remove('추억')
            owned.remove('추억의 펜던트')
            owned.remove('어느 날의 기억')
            consumed.append('추억의 펜던트')
            consumed.append('어느 날의 기억')
            value += 200
        else:
            print(f'재료가 부족합니다.')

    elif consum_item == '치성':
        if '억류된 찬송' in owned and '밀라르카' in owned:
            print(f'치성을 합성합니다.')
            owned.append('치성') # insert() 대신 append() 사용
            consums.remove('치성')
            owned.remove('억류된 찬송')
            owned.remove('밀라르카')
            consumed.append('억류된 찬송')

```

```

        if '무정전 전원장치' in owned and '미니어처 전봇대' in owned:
            print(f'인슐레이터를 합성합니다.')
            owned.append('인슐레이터') # insert() 대신 append() 사용
            consums.remove('인슐레이터')
            owned.remove('무정전 전원장치')
            owned.remove('미니어처 전봇대')
            consumed.append('무정전 전원장치')
            consumed.append('미니어처 전봇대')
            value += 200
        else:
            print(f'재료가 부족합니다.')
    else:
        print(f'보유중이거나 존재하지 않는 아이템 입니다.')

def menu_count_item(count_item):
    print(f'보유중 아이템 수 : {count_item}')
    print(f'보유중 아이템 가치 : {value}')
    print(f'소지금 : {money}')

def menu_end(count_item): # 매개변수로 count_item을 받도록 수정
    print(f'프로그램을 종료합니다.')
    # 0으로 나누는 에러 방지 처리
    if count_item > 0:
        star_light = ((value + money) // count_item) * 5
    else:
        star_light = 0
    return star_light

while True:
    print(f'-'*60)
    print(f'보유중 {owned}')
    print(f'미보유 {not_owned}')
    print(f'-'*60)

    print(f'실행할 항목을 선택')
    print(f'1. 시작 아이템 팩 선택')
    print(f'2. 아이템 등록')
    print(f'3. 아이템 판매')
    print(f'4. 아이템 합성')
    print(f'5. 아이템 목록 정보')
    print(f'6. 프로그램 종료')

```

```

menu = int(input(f'번호 입력 : '))
print(f'-'*60)

count_item = len(owned) # 오류 방지용

if menu == 1:
    menu_start_pack()

elif menu == 2:
    for i2 in range(3):
        add_item = input(f'등록할 아이템 이름 {i2 + 1} : ')
        if add_item in not_owned:
            value += float(input(f'아이템 가격 : '))
            not_owned.remove(add_item)
            owned.append(add_item)
        else:
            print(f'보유중이거나 존재하지 않는 아이템 입니다.')
            break

elif menu == 3:
    sell_item = input(f'판매할 아이템 이름 : ')
    menu_sell_item(sell_item)

elif menu == 4:
    consum_item = input(f'합성할 아이템 이름 : ')
    menu_consum_item(consum_item)

elif menu == 5:
    menu_count_item(count_item)

elif menu == 6:
    if money == 0 and value == 0:
        print(f'프로그램을 종료합니다.')
    else:
        result_light = menu_end(count_item) # 매개변수 전달
        print(f'얻은 별빛 {result_light}')
    break

else:
    print(f'잘못된 입력입니다.')
    continue

```

Git: From https://github.com/IE-PY-2026-1/ie-py-homework-chqhdpy
깃허브에 변경한 과제를 올리려고 하는데 이런 오류가 떠 무슨 오류야?

과제를 깃허브에 올리려다 문제가 생기셨군요. 올려주신 문구(`Git: From https://...`)는 깃허브 원격 저장소 주소를 확인하는 도입부 메시지입니다. 즉, 진짜 에러 내용이 담긴 뒷부분이 잘려 있는 상태입니다.

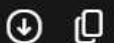
보통 저 줄 아래에 `error:`, `fatal:`, 또는 `hint:` 로 시작하는 상세한 원인이 적혀 있습니다.

정확한 에러 메시지를 보기 전이지만, 과제 저장소에 Push(업로드)할 때 가장 자주 발생하는 세 가지 에러와 해결책을 먼저 짚어드릴게요.

1. 원격 저장소의 내용이 로컬(내 컴퓨터)보다 최신일 때 (가장 흔함)

- 예상 에러 메시지: `error: failed to push some refs to...` 또는 `Updates were rejected because the remote contains work that you do not have locally.`
- 원인: 깃허브 저장소에 로컬 컴퓨터에는 없는 새로운 변경 사항이 있을 때 발생합니다.
- 해결 방법: 먼저 원격 저장소의 내용을 내 컴퓨터로 가져와서 병합(`pull`)한 뒤에 다시 올려야(`push`) 합니다.

Bash



```
git pull origin main # (또는 master)
git push origin main
```

2. 변경 사항을 확정(Commit)하지 않고 올리려 할 때

3. 로그인 및 권한 문제

- 예상 에러 메시지: `remote: Permission to ... denied to ...` 또는 `Authentication failed`
- 원인: 깃허브 로그인 토큰(Personal Access Token)이 만료되었거나, 해당 저장소에 쓰기 권한이 없는 계정으로 연동되었을 때 발생합니다.

현재 겪고 계신 상황을 정확히 파악해서 빠르게 해결해 드리고 싶습니다. `Git: From...` 문구 아래에 출력된 에러 메시지 전문을 전체 복사해서 보여주시겠어요?

👍 🗨 📄 ...

이게 전문이야

앗, '이게 전문이야'라고 말씀해 주셨는데, 깜빡하셨는지 댓글이나 본문에 에러 메시지 내용이 같이 안 올라온 것 같아요! 😊

에러가 난 부분의 텍스트를 복사해서 다시 한 번만 이곳에 붙여넣어 주시겠어요? 확인하는 대로 바로 어떤 문제인지, 어떻게 해결해야 하는지 알려드릴게요!

오류 메시지는 저게 끝이고 이건 git 로그야

```
2026-05-25 17:50:47.490 [info] > git log --oneline --cherry  
main...main@{upstream} -- [4ms]
```

```
2026-05-25 17:50:48.036 [info] > git pull --tags origin main  
[532ms]
```



로그를 올려주셔서 감사합니다. 원인이 명확하게 나왔네요!

발생한 핵심 오류는 `fatal: Need to specify how to reconcile divergent branches.` 입니다.

🔍 원인 분석: "분기된 브랜치(Divergent branches)"

쉽게 말해, 깃허브(원격 저장소)에 있는 내용과 현재 컴퓨터(로컬)에 있는 내용이 서로 다르게 업데이트되어 충돌이 예상되는 상태입니다.

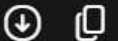
Git 입장에서는 "양쪽 다 고유한 변경 사항이 있는데, 이걸 어떤 방식으로 합칠지(병합할지) 네가 정해줘!"라고 묻고 멈춘 것입니다.

🔧 해결 방법

터미널(또는 VS Code 터미널)을 열고 아래 명령어를 순서대로 입력하면 해결됩니다. 초보자에게 가장 안전하고 일반적인 '기본 병합(Merge)' 방식을 지정해 주는 방법입니다.

1. Git에게 병합 방식 지정하기

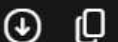
Bash



```
git config pull.rebase false
```

2. 깃허브의 최신 내용을 내 컴퓨터로 가져와서 합치기 (Pull)

Bash



```
git pull origin main
```

참고: 위 명령어를 입력했을 때 터미널 화면이 바뀌면서 커밋 메시지를 입력하라는 창(Vim 편집기)이 뜰 수 있습니다. 당황하지 마시고 키보드에서 `:wq` 를 타이핑한 뒤 `Enter` 를 누르시면 됩니다. (그대로 저장하고 종

참고: 위 명령어를 입력했을 때 터미널 화면이 바뀌면서 커밋 메시지를 입력하라는 창(Vim 편집기)이 뜰 수 있습니다. 당황하지 마시고 키보드에서 `:wq` 를 타이핑한 뒤 `Enter` 를 누르시면 됩니다. (그대로 저장하고 종료하겠다는 명령어입니다.)

3. 다시 내 과제를 깃허브로 올리기 (Push)

병합이 무사히 끝났다면, 원래 하려던 푸시를 진행합니다.

Bash



```
git push origin main
```

보통 학교 과제 저장소(GitHub Classroom 등)를 쓸 때, 깃허브 쪽에 교수님이 올려둔 피드백, 템플릿, Readme 파일 등의 변경 사항이 있는데 내 로컬 코드에는 그 내용이 반영되지 않았을 때 아주 흔하게 발생하는 에러입니다. 위 순서대로 진행하시면 깔끔하게 업로드될 것입니다!

